HW3 Single cycle CPU

2024084648 이종민

1. Impliment

1) CTRL unit

```
always @(*) begin
    //ininitailize to 0
    RegDst = 0; Jump = 0; Branch = 0; MemRead = 0; MemtoReg = 0;
    MemWrite = 0; ALUSrc = 0; SignExtend = 0; RegWrite = 0;
    ALUOp = 4'b0000; SavePC = 0; JR = 0;
```

이후에 킬 reg만 1로 바꾸기위해 초기에 0으로 초기화했다.

(1) R-Type

```
//R-Type instruction
    case (opcode)
        `OP_RTYPE: begin
            //JR
            if (funct == `FUNCT_JR) begin
                JR = 1;
            end
            else begin
                RegDst = 1;
                RegWrite = 1;
            end
            case (funct)
                `FUNCT_ADDU:    ALUOp = `ALU_ADDU;
                `FUNCT_AND:         ALUOp = `ALU_AND;
                `FUNCT_NOR:         ALUOp = `ALU_NOR;
                `FUNCT_OR:      ALUOp = `ALU_OR;
                `FUNCT_SLT:         ALUOp = `ALU_SLT;
                `FUNCT_SLTU:    ALUOp = `ALU_SLTU;
                `FUNCT_SUBU:    ALUOp = `ALU_SUBU;
                `FUNCT_XOR:         ALUOp = `ALU_XOR;
                `FUNCT_SLL:         ALUOp = `ALU_SLL;
                `FUNCT_SRA:         ALUOp = `ALU_SRA;
                `FUNCT_SRL:         ALUOp = `ALU_SRL;
            endcase
            end
```

R-type 명령어 분류하여 JR인 경우를 먼저 체크하고 이후 공통인 RegDst = 1과 regWrite = 1을 대입하고 funct에 맞게 ALUOp를 대입

했다.

(2) I-Type

```
//I-Type instruction
        `OP_ADDIU: begin
            ALUSrc = 1; SignExtend = 1; RegWrite = 1; ALUOp = `ALU_ADDU;
        end
        `OP_ANDI: begin
            ALUSrc = 1; RegWrite = 1;  ALUOp = `ALU_AND;
        end
        `OP_ORI: begin
            ALUSrc = 1; RegWrite = 1; ALUOp = `ALU_OR;
        end
        `OP_XORI: begin
            ALUSrc = 1; RegWrite = 1; ALUOp = `ALU_XOR;
        end
        `OP_LUI: begin
            RegWrite = 1; ALUSrc = 1; ALUOp = `ALU_LUI;
        end

        `OP_SLTI: begin
            SignExtend = 1; RegWrite = 1 ; ALUSrc = 1; ALUOp = `ALU_SLT;
        end
        `OP_SLTIU: begin
            SignExtend = 1; RegWrite = 1; ALUSrc = 1; ALUOp = `ALU_SLTU;
        end
        `OP_LW: begin
            SignExtend = 1; MemRead = 1; RegWrite = 1; MemtoReg = 1; ALUSrc = 1; ALUOp = `ALU_ADDU;
        end
        `OP_SW: begin
            SignExtend = 1; MemWrite = 1; ALUSrc = 1; ALUOp = `ALU_ADDU;
        end
        `OP_BEQ: begin
            SignExtend = 1; Branch = 1; ALUOp = `ALU_EQ;
        end
        `OP_BNE: begin
            SignExtend = 1; Branch = 1; ALUOp = `ALU_NEQ;
        end
```

I-type 명령어들의 reg들을 각각 명령어의 특성에 맞게 1로 바꾸어주고 ALUOp를 설정했다. 특히 OP_BEQ와 OP_BNE에서 SUBU 대신 EQ, NEQ를 사용했다.

(3) J-Type

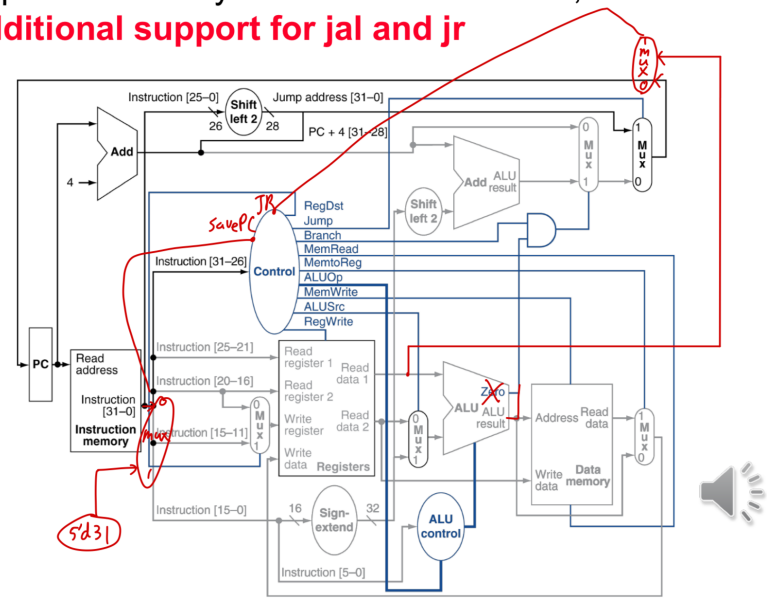
```
//J-Type instruction
    `OP_J: begin
        Jump = 1;
    end
    `OP_JAL: begin
        Jump = 1; RegWrite = 1; SavePC = 1;
    end
    default: begin

    end
```

명령어에 맞게 설정했다.

2) CPU

Implement what you learned in the class, but w/
additional support for jal and jr



설계도에 맞게 MUX는 삼항 연산자를 통해 구현하여 와이어를 전부 연결하고 모듈 인자들을 대입해 구현했다.

```
assign halt            = (inst == 32'b0);
assign ext_imm         = (SignExtend) ? {{16{immi[15]}}, immi} : {16'b0, immi};
assign opcode          = inst[31:26];
assign rs      = inst[25:21];
assign rt      = inst[20:16];
assign rd      = inst[15:11];
assign shamt   = inst[10:6];
assign funct   = inst[5:0];
assign immi    = inst[15:0];
assign immj    = inst[25:0];

//RF wire 연결
assign rd_addr1 = rs;
assign rd_addr2 = rt;


//ALU wire 연결
assign operand1 = rd_data1;
assign operand2 = ALUSrc ? ext_imm : rd_data2;

//MEM wire 연결
assign mem_addr = alu_result;
assign mem_write_data = rd_data2;
```

```
CTRL ctrl (
    //input
    .opcode(opcode),
    .funct(funct),
    //output
    .RegDst(RegDst),
    .Jump(Jump),
    .Branch(Branch),
    .JR(JR),
    .MemRead(MemRead),
    .MemtoReg(MemtoReg),
    .MemWrite(MemWrite),
    .ALUSrc(ALUSrc),
    .SignExtend(SignExtend),
    .RegWrite(RegWrite),
    .ALUOp(ALUOp),
    .SavePC(SavePC)
);

RF rf (
    //input
    .clk(clk),
    .rst(rst),
    //read related
    .rd_addr1(rd_addr1),
    .rd_addr2(rd_addr2),
    //write related
    .RegWrite(RegWrite),
    .wr_addr(wr_addr),
    .wr_data(wr_data),
    //output
    .rd_data1(rd_data1),
    .rd_data2 (rd_data2)
);

MEM mem (
    //instmem input
    .clk(clk),
    .rst(rst),
    .inst_addr(PC),
    //instmem output
    .inst(inst),

    //datamem input
    .mem_addr(mem_addr),
    .MemWrite(MemWrite),
    .mem_write_data(mem_write_data),
    //datamem output
    .mem_read_data(mem_read_data)
);

ALU alu (
    //input
    .operand1(operand1),
    .operand2(operand2),
    .shamt(shamt),
    .funct(ALUOp),
    //output
    .alu_result(alu_result)
);
```

설계도와 다른 부분 :

(1) 컨트롤 유닛의 JR 신호를 통해 만약 JR이 1이면 rd_data1로 PC를 점프함

```
PC_next = JR ? rd_data1 : Jump ?  {PC_plus4[31:28], immj, 2'b00} :
            (Branch && alu_result) ?  (PC_plus4 + (ext_imm << 2)) : PC_plus4;
```

(2) JAL일 때 1인 SavePC를 통해 링크할 레지스터 주소를 받고 write 할 data를 선택함.

```
wr_addr = SavePC ? 5'd31 : (RegDst ? rd : rt);
wr_data = SavePC ? PC_plus4 : (MemtoReg ? mem_read_data : alu_result);
```

(3) ALU가 EQ과 NEQ를 지원하기 때문에 Zero 신호 대신 alu result를 똑같이 사용함.

```
PC_next = JR ? rd_data1 : Jump ?  {PC_plus4[31:28], immj, 2'b00} :
            (Branch && alu_result) ?  (PC_plus4 + (ext_imm << 2)) : PC_plus4;
```

(4) Pc+4를 저장할 와이어를 이후 인덱스 연산을 위해 추가함.

```
// Define PC
reg [31:0]  PC;
reg [31:0]  PC_next;
wire [31:0]  PC_plus4 = PC + 32'd4;
```